EXPLORATORY DATA ANALYSIS

Chapter 23

Designing a Reproducible EDA Workflow

From one-off analysis to a process that can be inspected, rerun, checked, and maintained.

Python • NumPy • Pandas • Matplotlib (object-oriented API) • Seaborn

1. Learning Objectives

• Explain reproducibility, repeatability, and auditability.

• Design a project structure that separates raw inputs, code, outputs, and documentation.

• Record data provenance, versions, filters, exclusions, and analytical assumptions.

• Use configuration, stable paths, dependency records, and controlled randomness.

• Validate data at intake and preserve evidence when checks fail.

• Organize analysis into reusable functions and cleanly executable notebooks or scripts.

• Save tables and figures without relying on hidden notebook state.

• Use Git responsibly while protecting private data and credentials.

2. Review of Earlier Chapters

This review revisits the foundations needed for a reproducible workflow. Chapter 22, the immediately previous chapter, is reviewed separately in Section 3.

• Chapters 1–3: EDA is iterative investigation; understand the observational unit, metadata, variable meaning, and data quality before interpretation.

• Chapters 4–9: numerical and categorical summaries, time variables, distributions, quantiles, skewness, and kurtosis depend on explicit definitions and denominators.

• Chapters 10–12: visual choices, scales, bins, facets, and encodings affect interpretation; figures need enough context to be understood later.

• Chapters 13–15: association, covariance, dependence, and grouped analysis require clear variable types, group definitions, and sample sizes.

• Chapters 16–17: unusual observations must be investigated; a flagged row is not automatically an error or a row to delete.

• Chapter 18: high-dimensional analysis needs deliberate handling of redundancy, missingness, multiple comparisons, and computational limits.

• Chapter 19: time-series analysis requires explicit time zones, ordering, cadence, and aggregation rules.

• Chapter 20: sampling, missingness, and collection bias limit what can be generalized.

• Chapter 21: automated profiling is useful for screening, but warnings need human interpretation and recorded settings.

3. Review of Chapter 22 — EDA for Different Data Domains

Chapter 22 emphasized that the domain changes the meaning of a row, a variable, and a useful denominator. Customer records, transactions, education records, clinical-style data, product analytics, logs, surveys, text, and geospatial data each need domain-aware checks.

Expected reasoning for the previous assignment/project: a strong case study states the domain, observational unit, intended question, data source, privacy constraints, important quality risks, suitable summaries, and limitations. It does not apply a generic chart or metric without explaining why it fits.

Example: a transaction dataset may have one row per transaction line rather than one row per customer. A customer-level question therefore requires deliberate aggregation. A time-based log may have missing intervals because no event occurred, collection failed, or the system was offline. Those explanations are not interchangeable.

4. What Does Reproducible EDA Mean?

Reproducibility means another person—or you later—can use the recorded inputs, code, settings, and environment to obtain the same or acceptably equivalent analytical results. **Repeatability** is the ability to rerun the same analysis under

the same conditions and obtain consistent results. **Auditability** means a reviewer can trace where data came from, what changed or was excluded, which methods were used, and how conclusions were reached.

Saving a notebook is not enough. A notebook can contain hidden state, manually edited cells, hard-coded local paths, unrecorded downloads, and outputs generated from an older dataset.

Result = f(data, code, configuration, environment, execution conditions)

4.1 Exact sameness is not always possible

• A fixed dataset and deterministic operations should normally produce the same outputs.

• Random sampling, approximate algorithms, parallel computation, or external services may introduce variation. Record seeds and settings where possible.

• A source website or API may change after download. Keep a permitted snapshot or stable source identifier and extraction timestamp.

• Floating-point operations can differ slightly across hardware or library versions. Use reasonable numerical tolerances where exact equality is inappropriate.

5. A Professional EDA Project Structure

A project should be navigable without requiring the author to explain where everything is. Adapt this structure to the project's size:

```
eda_project/
README.md
pyproject.toml
.gitignore
config/
analysis_config.toml
data/
raw/
interim/
processed/
notebooks/
01_data_intake.ipynb
02_quality_and_profile.ipynb
03_analysis.ipynb
src/
eda_project/
io.py
validation.py
summaries.py
plots.py
outputs/
tables/
figures/
reports/
logs/
docs/
data_dictionary.md
decisions.md
```

• data/raw/: original received or downloaded data; treat as read-only.

• data/interim/: useful intermediate artifacts.

• data/processed/: derived datasets created by documented steps.

• notebooks/: exploratory narrative, code, outputs, and explanations.

• src/: reusable functions rather than repeated notebook code.

• config/: changeable paths, column names, date ranges, and thresholds.

• outputs/: generated tables, figures, and reports.

• logs/: execution messages and audit information; avoid sensitive row-level data.

• docs/: data dictionary, methodology, assumptions, and decision history.

For a small assignment, this complete structure may be unnecessary. The principle is separation of responsibilities, not creating folders for decoration.

6. Data Intake, Preservation, and Provenance

6.1 Preserve the original input

Never overwrite the only copy of source data. Load it, inspect it, and create derived outputs separately. If the source contains a mistake, preserve the original and document any correction.

6.2 Record provenance

Provenance answers: where did the dataset come from, when was it obtained, which version was used, and what happened before analysis?

• Source name/provider and source location.

• Download or extraction timestamp and source publication date, if available.

• Filename, version, and checksum/hash.

• File format, encoding, delimiter, and parsing options.

• Filters, date ranges, joins, exclusions, and deduplication rules.

• Row and column counts at intake and after important steps.

• Responsible analyst/team, subject to privacy rules.

6.3 Create a file checksum

A cryptographic hash helps identify whether file bytes changed. It does not prove that a dataset is correct; it is a fingerprint, not a quality certificate.

```
from pathlib import Path
import hashlib

def sha256_file(path, chunk_size=1024 * 1024):
digest = hashlib.sha256()
with Path(path).open("rb") as file:
while chunk := file.read(chunk_size):
digest.update(chunk)
return digest.hexdigest()

source_path = Path("data/raw/sales.csv")
print(sha256_file(source_path))
```

7. Configuration and Stable Paths

Avoid scattering machine-specific paths and analysis choices throughout a notebook. Put changeable values in one configuration file.

```
from pathlib import Path
import tomllib

PROJECT_ROOT = Path.cwd()

with (PROJECT_ROOT / "config" / "analysis_config.toml").open("rb") as
    file:
config = tomllib.load(file)

data_path = PROJECT_ROOT / config["data"]["input_file"]
output_dir = PROJECT_ROOT / config["outputs"]["directory"]
output_dir.mkdir(parents=True, exist_ok=True)
```

Example TOML configuration:

```
[data]
input_file = "data/raw/sales.csv"
date_column = "order_date"

[analysis]
start_date = "2025-01-01"
end_date = "2025-12-31"
random_seed = 42

[outputs]
directory = "outputs"
```

Python's built-in tomllib is available in Python 3.11+. For older versions, use a suitable TOML reader or another configuration format. A notebook's current working directory can differ depending on how it is launched, so verify it.

7.1 Avoid hidden configuration

• Do not hard-code a personal desktop path in a shared project.

• Do not hide important filters inside an easily overlooked cell.

• Do not store passwords, API keys, or tokens in notebooks or Git.

• Do not commit private customer, student, or health records to a public repository.

• Use environment variables or an approved secret manager for credentials.

8. Environment and Dependency Management

A notebook that works on one computer may fail on another because library versions differ. Record Python and important package versions.

```
import sys
import pandas as pd
import numpy as np
import matplotlib
import seaborn as sns

environment = {
"python": sys.version,
"pandas": pd.__version__,
"numpy": np.__version__,
"matplotlib": matplotlib.__version__,
"seaborn": sns.__version__,
}

for package, version in environment.items():
print(f"{package}: {version}")
```

For a small project, a requirements file or pyproject.toml may be enough. For stronger reproducibility, use a lock file from your environment manager. Pinning improves repeatability but needs maintenance and security updates.

9. Validation Before Analysis

A reproducible workflow should fail early when input does not meet essential expectations. Validation is a guardrail, not a replacement for EDA.

9.1 Basic schema and value checks

```
import pandas as pd

df = pd.read_csv(data_path)

required_columns = {"order_id", "order_date", "amount"}
missing_columns = required_columns.difference(df.columns)

if missing_columns:
raise ValueError(
f"Required columns are missing: {sorted(missing_columns)}"
)

if df["order_id"].isna().any():
raise ValueError("order_id contains missing values")

if (df["amount"].dropna() < 0).any():
raise ValueError("Negative amounts require investigation")

df["order_date"] = pd.to_datetime(
df["order_date"],
errors="coerce"
)

parse_failures = df["order_date"].isna().sum()
print("Date values unavailable after parsing:", parse_failures)
```

The negative-amount rule is domain-dependent: refunds or reversals may legitimately be negative. Validation rules must come from the data dictionary and context, not habit.

9.2 Errors versus investigation flags

Use a hard error for a condition that must be true before analysis proceeds, such as a required identifier column being absent. Use a warning or investigation flag for unusual but possibly valid values, such as a very large transaction. Never quietly drop rows because a check failed.

10. Modular Analysis: Functions with Clear Responsibilities

A long notebook full of repeated code is difficult to maintain. Move stable operations into functions with clear purposes, explicit inputs, predictable outputs, and minimal hidden dependencies.

```
def numeric_profile(data, columns):
available = [col for col in columns if col in data.columns]
numeric = data[available].select_dtypes(include="number")

summary = numeric.describe().T
summary["missing_count"] = numeric.isna().sum()
summary["missing_rate"] = numeric.isna().mean()

return summary

profile = numeric_profile(df, columns=["amount"])
profile.to_csv("outputs/numeric_profile.csv")
```

A more complete function should define behavior for empty selections, non-numeric columns, duplicate labels, and output naming.

10.1 Separate responsibilities

• Loading: read the source with explicit parsing options.

• Validation: check required structure and domain constraints.

• Analysis: calculate summaries and investigate patterns.

• Presentation: create tables, figures, and written findings.

• Export: save outputs with traceable names and metadata.

11. Notebook Hygiene and Clean Execution

Jupyter combines code, outputs, explanations, and visual evidence. But visible cell order can differ from actual execution order.

• Restart the kernel and run all cells from top to bottom before delivery.

• Remove stale outputs that no longer match current code or data.

• Do not rely on variables created by a cell that appears later.

• Keep imports and configuration near the beginning.

• Explain important decisions in Markdown cells.

• Avoid enormous printed tables; show useful samples and save full results when needed.

• Use meaningful headings and a clear analysis sequence.

• Record dataset version and execution date.

A clean-kernel run is one of the strongest practical checks for notebook reproducibility.

11.1 Randomness and sampling

```
sample = df.sample(
n=min(5000, len(df)),
random_state=42
)
```

A fixed random_state makes this sampling step repeatable for the same data and software behavior. It does not make a changing source dataset stable.

11.2 Matplotlib object-oriented plotting

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.hist(df["amount"].dropna(), bins="fd", edgecolor="white")
ax.set(
title="Distribution of transaction amounts",
xlabel="Amount",
ylabel="Number of transactions"
)
```

```
fig.tight_layout()
fig.savefig(
"outputs/amount_distribution.png",
dpi=160,
bbox_inches="tight"
)
plt.close(fig)
```

The explicit figure and axes objects make plots easier to customize and reproduce. Saving to a predictable location and closing the figure are useful in scripts and repeated plotting workflows.

12. Logging and Run Metadata

A run record should tell a future reader what happened without exposing sensitive data. Record high-level events, counts, warnings, and output locations—not private row contents.

```
import logging
from pathlib import Path

log_dir = Path("logs")
log_dir.mkdir(exist_ok=True)

logging.basicConfig(
filename=log_dir / "eda_run.log",
level=logging.INFO,
format="%(asctime)s | %(levelname)s | %(message)s",
force=True
)

logging.info("EDA run started")
logging.info("Input file: %s", data_path.name)
logging.info("Rows loaded: %d", len(df))
logging.info("Columns loaded: %d", len(df.columns))
logging.info("Missing amount values: %d", df["amount"].isna().sum())
```

A structured metadata record can include:

```
from datetime import datetime, timezone
import json

run_metadata = {
"run_time_utc": datetime.now(timezone.utc).isoformat(),
"input_file": str(data_path),
"input_sha256": sha256_file(data_path),
"rows_loaded": int(len(df)),
"columns_loaded": int(len(df.columns)),
"python": sys.version.split()[0],
"pandas": pd.__version__,
"numpy": np.__version__,
}

metadata_path = Path("outputs/run_metadata.json")
metadata_path.parent.mkdir(parents=True, exist_ok=True)
metadata_path.write_text(
json.dumps(run_metadata, indent=2),
encoding="utf-8"
)
```

For client work, consider whether file paths, source identifiers, timestamps, or metadata reveal confidential information. Share only what is authorized.

13. Git and Version Control

Git records changes to code and documentation. It helps answer what changed, when, and why. It is not a substitute for backups or data governance.

• Commit source code, configuration examples, documentation, and permitted small sample data.

• Use meaningful commit messages, such as 'Add missingness audit by column'.

• Keep private datasets, credentials, caches, and large outputs out of public Git.

• Use .gitignore and inspect staged files before committing.

• Record important analytical decisions in documentation or commit messages.

• Deleting a secret in a later commit does not necessarily remove it from Git history.

```
# Example .gitignore entries

__pycache__/
.ipynb_checkpoints/
.venv/
.env
```

```
data/raw/
data/interim/
logs/
outputs/
*.parquet
*.csv
```

This is only an example. You may want to version selected small outputs or public sample data. Decide deliberately what belongs in version control.

14. Decisions, Exclusions, and Analytical Audit Trail

Record decisions that could affect the result:

• Why columns were excluded from a particular analysis.

• How duplicate records were identified and handled.

• Which date range and timezone were used.

• Whether missing values were excluded, retained, or summarized separately.

• How groups and denominators were defined.

• Which outliers were investigated and what was concluded.

• Which records were filtered and how many were affected.

• Why a chart, statistic, or association measure was selected.

A simple decision log:

```
| Date | Decision | Reason | Impact |
|---|---|---|---|
| 2026-11-14 | Excluded test orders | Confirmed by source owner | 42 rows
    |
| 2026-11-14 | Kept negative amounts | Refunds are valid | None removed |
| 2026-11-14 | Used UTC timestamps | Source timestamps are UTC | Dates
   aligned |
```

'The distribution has a long right tail' is an observation. 'We removed the largest 1% of values' is an analytical decision and needs a reason, impact assessment, and record.

15. Saving Tables, Figures, and Reports

• Use descriptive filenames, such as numeric_profile.csv or monthly_order_counts.png.

• Include analysis date or version when multiple runs must be compared.

• Save machine-readable tables as CSV or Parquet where appropriate.

• Save plots with titles, units, labels, and sample context.

• Avoid overwriting important outputs without a versioning policy.

• Keep a short output index in the README or report.

For tables, include denominators and definitions where counts or percentages could be ambiguous. For figures, include population, time window, and important filters in captions or report text.

16. A Practical End-to-End Workflow

1. Create the project folder and write a short README.

2. Record source, extraction date, file version, and checksum.

3. Load original data without modifying it.

4. Validate schema, types, keys, and essential domain constraints.

5. Profile completeness, uniqueness, distributions, and coverage.

6. Clarify the observational unit and define populations and denominators.

7. Perform quantitative and visual EDA using documented choices.

8. Investigate unusual values and record decisions; do not delete automatically.

9. Create tables and figures through reusable functions.

10. Save outputs, logs, metadata, and concise findings.

11. Restart and run the notebook from a clean kernel.

12. Review privacy, Git contents, filenames, and delivery completeness.

The order is a guide, not a rigid one-way pipeline. EDA is iterative: an unexpected pattern may send you back to inspect quality, source context, or a variable definition.

17. Reproducibility Checklist

• README explains the project and how to run it.

• Original input is preserved; source and version are recorded.

• Observational unit and important variable definitions are documented.

• Filters, exclusions, and denominators are explicit.

• Paths and analysis settings are centralized or clearly declared.

• Python and important package versions are recorded.

• Random operations use recorded seeds where appropriate.

• Validation checks run before analysis.

• Reusable functions have clear inputs and outputs.

• Notebook runs from a clean kernel in top-to-bottom order.

• Figures, tables, logs, and metadata are saved predictably.

• No unauthorized personal data or secrets are included.

• Important decisions and limitations are documented.

18. Common Mistakes

• Saving only the notebook: data, dependencies, settings, and source details may be missing.

• Overwriting raw data: original evidence is lost and changes cannot be traced.

• Hard-coded local paths: another machine cannot locate the files.

• Hidden notebook state: cells run only because of undocumented execution history.

• Unrecorded filters: the final sample cannot be reconstructed.

• Uncontrolled randomness: sampled rows or approximate results may change.

• Overengineering: an unnecessarily complex structure can make a small project harder to use.

• Logging private rows: diagnostic output can become a privacy leak.

• Treating a checksum as proof of quality: a stable file can still contain incorrect data.

• Assuming reproducibility guarantees correctness: reproducible mistakes are still mistakes.

19. Summary

• Reproducible EDA makes data, code, configuration, environment, and decisions traceable.

• Preserve raw data and create derived outputs separately.

• Record provenance, checksums, versions, filters, exclusions, and row counts.

• Use validation checks, modular functions, clean notebook execution, and controlled randomness.

• Save figures and tables with clear labels and definitions.

• Use Git for code and documentation while protecting data and secrets.

• Reproducibility improves trust and maintainability, but it does not guarantee correctness.

20. Assignment

Part A — Concepts

• Explain reproducibility, repeatability, and auditability in your own words.

• Why should raw data be separate from processed data?

• What is the difference between a validation error and an investigation warning?

• Why does a fixed random seed not make an analysis fully reproducible?

• Give three examples of analytical decisions that should be documented.

• Why might a notebook that runs on your computer fail for another analyst?

Part B — Workflow design You receive a CSV containing online orders: order identifier, customer identifier, order timestamp, product category, quantity, and order amount. Design a project structure and README outline. Identify at least five intake checks and five provenance fields.

Part C — Critical thinking A colleague reports that average order amount increased after excluding unusually large orders. What should be recorded before accepting this result? Explain how exclusion may change the question being answered.

Part D — Practical task Create a configuration file, load data using a relative project path, calculate a numeric profile, save one Matplotlib OO figure, and record the input hash and package versions. Restart the kernel and run all cells from the beginning.

21. Coding Project — Reproducible EDA Project Template

Objective: turn one of your earlier EDA datasets into a small project that another analyst can inspect and rerun.

Requirements

• Create a project folder with README, config, data, notebooks, source code, outputs, and docs.

• Preserve the raw dataset and record its source, date, row count, and SHA-256 hash.

• Create configuration for input path, important column names, date range, and random seed.

• Write validation functions for required columns, missing identifiers, data types, and at least two domain-specific rules.

• Build reusable functions for numerical and categorical summaries and at least two plots.

• Save tables and figures in predictable output folders.

• Create run metadata and a log with key row counts and warnings.

• Document filters, exclusions, unusual observations, and analytical decisions.

• Use Git for code and documentation; do not commit private data or secrets.

• Run the notebook from a clean kernel and document the execution procedure in README.

Deliverables: project folder or repository, README, configuration example, notebook, reusable Python module(s), output figures/tables, run metadata, and a short limitations note.

Master notebook instruction: continue extending your single cumulative EDA master notebook. Integrate this chapter's reproducibility checks, provenance record, and reusable workflow patterns into that evolving notebook rather than

creating an unrelated permanent notebook for every chapter.

Evaluation: project clarity, validation correctness, traceability, clean execution, quality of summaries/figures, privacy, and delivery hygiene.

22. Preview of Next Chapter

Chapter 24 — Building a Professional EDA Report

The next chapter turns analysis outputs into a polished report. We will organize an executive summary, describe the dataset and methodology, present data-quality findings, choose and caption tables and figures, explain limitations, and make recommendations without overstating what the data proves.

Chapter 23 makes the work reproducible. Chapter 24 will make the work understandable and useful to the person who commissioned it.